

Roll Number

--	--	--	--	--	--	--

0	0	0	0	0	0	0
1	1	1	1	1	1	1
2	2	2	2	2	2	2
3	3	3	3	3	3	3
4	4	4	4	4	4	4
5	5	5	5	5	5	5
6	6	6	6	6	6	6
7	7	7	7	7	7	7
8	8	8	8	8	8	8
9	9	9	9	9	9	9

The OMR Sheet will be computer checked. Fill the circles completely and dark enough for proper detection. Use Dark HB pencil or ballpen (black or blue) for marking. While erasing, erase properly.

Avoid Improper Marking



Partially Filled



Lightly filled



Name: _____

Roll No.: _____

Problem 1. [10 points] In class, we saw an $O(n \log n)$ time algorithm to count the number of inversions in an array $A[1 : n]$. Recall, an inversion is a pair (i, j) with $1 \leq i < j \leq n$ and $A[i] > A[j]$. Let us call an inversion (i, j) bounded if $A[j] < A[i] \leq A[j] + 10$. That is, $A[i]$ is bigger than $A[j]$ but not by much. Design an $O(n \log n)$ time algorithm to count the number of bounded inversions. You may assume that the numbers in the array A are distinct.

(a) **[2 points]** In 2-3 sentences describe how your algorithm is different from counting inversions as done in lectures.

Answer: Instead of simply adding the count of all remaining elements in the left half whenever a smaller element is pulled from the right half—this algorithm requires a more targeted counting mechanism during the merge phase. Specifically, it uses an additional sliding window over the sorted left half to exclusively count elements that satisfy the strict boundary condition $A[i] \leq A[j] + 10$ for each element $A[j]$ in the right half. This bounded counting step is executed alongside the standard array merging process, allowing it to run in $O(n \log n)$ time complexity.

Rubric: +1.0 for identifying the mechanism: The student mentions replace the standard counting step with a more targeted technique, specifically identifying the use of a "sliding window" or "two/multiple pointers" or "10 condition checks". +1.0 for bounded condition Check: The student explains that this window scans the sorted left half to exclusively count elements that fall within the strict boundary $A[i] \leq A[j] + 10$ for each element $A[j]$ in the right half.

Note on partial credit: Award partial credit +1.0 if the explanation is conceptually on the right track but vague (e.g., "we check the +10 condition during the merge" without explaining how it's done efficiently).

(b) **[6 points]** Write a pseudocode of your algorithm.

Algorithm 1 Count Bounded Inversions

```

1: procedure COUNTBOUNDEDINVERSIONS( $A, n$ )
2:   if  $n \leq 1$  then                                     ▷ Base case
3:     Return  $(0, A)$ 
4:   end if
5:    $mid = \lfloor n/2 \rfloor$                                        ▷ Divide array
6:    $(left\_count, L) = \text{COUNTBOUNDEDINVERSIONS}(A[1 \dots mid], mid)$ 
7:    $(right\_count, R) = \text{COUNTBOUNDEDINVERSIONS}(A[mid + 1 \dots n], n - mid)$ 
8:    $(split\_count, merged\_A) = \text{MERGEANDCOUNT}(L, R, mid, n - mid)$    ▷ Merge and count
9:   Return  $(left\_count + right\_count + split\_count, merged\_A)$ 
10: end procedure

```

Rubric: Divide and Conquer Structure +2.0. +0.5 Base Case: Correctly handles the base case (e.g., returning 0 inversions for arrays of size ≤ 1). +0.5 for Divide Step: Correctly splits the array into two halves. +1.0 for Recursive Calls and merge: Makes recursive calls to both halves and correctly sums left_count, right_count, and split_count.

Merge and Count Logic +4.0. -1.0 if merge and count is more than $O(n)$.

(c) **[2 points]** Argue runtime.

Solution The divide-and-conquer structure yields the standard recurrence $T(n) = 2T(n/2) + O(n)$. The combine step runs in $O(n)$ time because both pointers strictly advance through the left array without ever resetting, resulting in at most $O(|L| + |R|)$ total operations. By the Master Theorem, this resolves to an overall time complexity of $O(n \log n)$.

Rubric: +0.5 for correct recurrence. +1.0 for correctly arguing why their combine step is $O(n)$. +0.5 for final correct claim $O(n \log n)$.

Problem 2. [20 points] You are given a set of n *divisible* items - the size of item i is denoted by s_i while the non-negative profit that you can earn from item i is p_i . Further, you are given a knapsack of size W . The goal is to pick items - this

Algorithm 2 Merge And Count Bounded Inversions

```
1: procedure MERGEANDCOUNT( $L, R, n_1, n_2$ )
2:    $count = 0$ 
3:    $start\_ptr = 1$ 
4:    $end\_ptr = 1$ 
5:   for  $j = 1$  to  $n_2$  do ▷ Sliding window
6:     while  $start\_ptr \leq n_1$  and  $L[start\_ptr] \leq R[j]$  do ▷ Find lower bound
7:        $start\_ptr = start\_ptr + 1$ 
8:     end while
9:     while  $end\_ptr \leq n_1$  and  $L[end\_ptr] \leq R[j] + 10$  do ▷ Find upper bound
10:       $end\_ptr = end\_ptr + 1$ 
11:    end while
12:     $count = count + (end\_ptr - start\_ptr)$  ▷ Add valid elements
13:  end for
14:   $merged\_A = \text{STANDARDMERGE}(L, R, n_1, n_2)$  ▷ Standard merge
15:  Return ( $count, merged\_A$ )
16: end procedure
```

time even fractionally - so that (a) The total (possibly fractional) size of items should not exceed the size of the knapsack and (b) the total profit is maximized. You may assume that the profit of a q fraction ($q \leq 1$) of item i is $q \cdot p_i$.

- (a) [10 points] There is a natural greedy algorithm for this problem. Sort the items according to non-increasing p_i/s_i ratio. Then keep picking the items according to the above ordering as long as they fit, using a fractional item when necessary. Prove by a formal exchange argument that this greedy algorithm always produces the optimal solution for fractional knapsack.

Solution.

Let G be the greedy solution and let O be any optimal solution. We show that $\text{profit}(G) \geq \text{profit}(O)$.

Assume items are labeled so that

$$\frac{p_1}{s_1} \geq \frac{p_2}{s_2} \geq \dots \geq \frac{p_n}{s_n}.$$

Let $g_i \in [0, 1]$ be the fraction of item i taken by the greedy algorithm, and $o_i \in [0, 1]$ be the fraction taken by the optimal solution.

By construction of the greedy algorithm, there exists an index k such that $g_i = 1$ for $i < k$, $g_k \in [0, 1]$, and $g_i = 0$ for $i > k$. That is, greedy fills the knapsack by taking items in decreasing ratio order.

Exchange step. Suppose $O \neq G$. Let i be the smallest index with $o_i \neq g_i$. There are two cases.

- **Case 1:** $o_i < g_i$. Greedy takes more of item i than the optimal does. Since O uses the full capacity W (taking more can only help profit), the “missing” fraction $(g_i - o_i) \cdot s_i$ of item i must be compensated by extra fractions of items $j > i$ in O (items $j < i$ have $o_j = g_j$ by choice of i).

We increase o_i by δ/s_i and decrease o_j by δ/s_j for some $j > i$ with $o_j > g_j$, keeping total size the same. The change in profit is:

$$\delta \cdot \frac{p_i}{s_i} - \delta \cdot \frac{p_j}{s_j} \geq 0$$

since $j > i$ implies $\frac{p_i}{s_i} \geq \frac{p_j}{s_j}$. So profit does not decrease.

- **Case 2:** $o_i > g_i$. This cannot happen. Since $g_j = 1 = o_j$ for all $j < i$, the remaining capacity when greedy reaches item i equals the remaining capacity in O after items $1, \dots, i-1$. Greedy takes as much of item i as possible, so $o_i > g_i$ would violate the capacity constraint.

By repeatedly applying this exchange—each step makes O agree with G on one more item without decreasing profit—we transform O into G with $\text{profit}(G) \geq \text{profit}(O)$. Hence greedy is optimal.

Rubric:

- **Setup and notation (2 points).** The solution clearly defines the greedy solution G and an arbitrary optimal solution O , and states what “greedy order” means (sorting by p_i/s_i).
Partial credit (1 pt): setup is present but imprecise (e.g. no notation for fractions taken).
- **Identifying the exchange (3 points).** The solution identifies a concrete difference between O and G —some item i where $o_i < g_i$ —and describes how to fix it by shifting capacity from a lower-ratio item j to item i . Full marks require that the exchange explicitly preserves feasibility (total size $\leq W$).
Partial credit (1–2 pts): exchange is vaguely described or does not explicitly maintain feasibility.
- **Profit does not decrease (3 points).** The solution argues that the exchange does not decrease profit, using the fact that $p_i/s_i \geq p_j/s_j$. The inequality must be clearly stated and connected to the sorting order.
Partial credit (1–2 pts): direction of inequality is correct but argument is hand-wavy.
- **Convergence and conclusion (2 points).** The solution argues that repeated exchanges transform O into G , and concludes that $\text{profit}(G) \geq \text{profit}(O)$.
Partial credit (1 pt): conclusion is stated but termination of the exchange process is not addressed.

(b) A naive implementation of the above algorithm runs in time $O(n \log n)$ since the work will be dominated by the sorting subroutine. Now design a *linear time* (yes, its $O(n)$) implementation of the above algorithm (Hint : Divide and Conquer; you have seen a very similar problem recently)

- (i) **[3 points]** In 4-5 sentences describe the high level idea.

The key insight is that we don’t need a full sort — we only need to find which items are fully taken, which is the “split” item (partially taken), and which are not taken at all. We use the median-of-medians linear-time selection algorithm (like quickselect) to partition items around the median of their profit-to-size ratios. After partitioning, we compute the total size of items in the “high ratio” half. If that total size is less than or equal to W , we take all those items, subtract their total size from W , and recurse on the remaining items. If the total size exceeds W , we recurse on just the high-ratio half with the same W . This is analogous to the linear-time selection algorithm and gives us a recurrence $T(n) = T(n/2) + O(n)$, which solves to $O(n)$.

- (ii) **[5 points]** Write a pseudocode of your algorithm.

Algorithm 3 Linear-Time Fractional Knapsack

```
1: procedure FRACKNAPSACK( $items, W$ )
2:   if  $W = 0$  or  $|items| = 0$  then
3:     return 0
4:   end if
5:   if  $|items| = 1$  then
6:     return  $\min(W, s_1) \cdot (p_1/s_1)$ 
7:   end if
8:    $m \leftarrow \text{MEDIANOFMEDIANS}(\{p_i/s_i : i \in items\})$   $\triangleright O(n)$  median selection
9:    $H \leftarrow \{i \in items : p_i/s_i > m\}$   $\triangleright$  high-ratio items
10:   $M \leftarrow \{i \in items : p_i/s_i = m\}$   $\triangleright$  equal-to-median items
11:   $L \leftarrow \{i \in items : p_i/s_i < m\}$   $\triangleright$  low-ratio items
12:   $S_H \leftarrow \sum_{i \in H} s_i$ 
13:  if  $S_H \geq W$  then
14:    return FRACKNAPSACK( $H, W$ )  $\triangleright$  answer lies within  $H$ 
15:  else
16:     $profit \leftarrow \sum_{i \in H} p_i$ 
17:     $W' \leftarrow W - S_H$ 
18:     $S_M \leftarrow \sum_{i \in M} s_i$ 
19:    if  $S_M \geq W'$  then
20:      return  $profit + W' \cdot m$   $\triangleright$  fill remainder with median-ratio items
21:    else
22:       $profit \leftarrow profit + \sum_{i \in M} p_i$ 
23:       $W'' \leftarrow W' - S_M$ 
24:      return  $profit + \text{FRACKNAPSACK}(L, W'')$ 
25:    end if
26:  end if
27: end procedure
```

Rubric:

- +0.5 for a correct base case
- +1 for line 8 (finding the median. you must specify quickselect or median-of-median or linear time selection etc.)
- +1.5 for correctly creating the three categories H, M, L (or something analogous)
- +2 for each of the recursive calls under correct conditions.

Remark. The way I have written the pseudocode will return the maximum profit rather than set of items. You may also write the algorithm to return the actual (fractional) set of items. Further, if your high level idea does not match the pseudocode, you get a 0.

- (iii) **[2 points]** Argue runtime. At each level of recursion we perform $O(n)$ work: the median-of-medians selection is $O(n)$, and partitioning plus computing sums are each $O(n)$. We recurse on *at most one* of H or L , each of size at most $\lceil n/2 \rceil$ (M is handled directly implicitly). This gives the recurrence:

$$T(n) = T(n/2) + O(n).$$

Expanding: $T(n) = O(n + n/2 + n/4 + \dots) = O(n)$ by the geometric series. Equivalently, by the Master Theorem with $a = 1, b = 2, f(n) = \Theta(n)$, we obtain $T(n) = O(n)$.

Rubric: +1 for correctly setting up the recurrence and +1 for the correct solution. If pseudocode is wrong, you get 0 for this part.

Problem 3. Upon time traveling to an ancient laboratory, you find n drops of a *mystical* liquid arranged in an order. The *volume* of each drop is given to you. You find a scroll with the following instructions.

You can do the following operation as many times as you like until just one drop remains:

Combine any two *adjacent* drops and place the combined drop at the same place in the order

When you combine two drops of volumes x and y respectively, you get a new drop whose volume is the sum $x + y$ of the two original volumes (i.e. volume is conserved). Also, you gain an amount of energy for your time traveling device equal to $x^2 + y^2$.

For example,

- (i) Start with volumes

1 4 2 3 5

- (ii) Combine the second and the third drop to get volumes

1 6 3 5

and gain energy $4^2 + 2^2 = 20$

- (iii) Combine the third and the fourth drop to get volumes

1 6 8

and gain energy $3^2 + 5^2 = 34$

- (iv) Combine the first and the second drop to get volumes

7 8

and gain energy $1^2 + 6^2 = 37$

- (v) Combine the remaining two drops to get volumes

15

and gain energy $7^2 + 8^2 = 113$

Hence, you gain a total energy of $20 + 34 + 37 + 113 = 204$.

Eager to time travel back to 2026, you would like to maximize the energy gained for your time traveling device. Give a polynomial time algorithm to find the maximum energy that you can generate using a valid sequence of operations.

- (a) [1 points] Define the subproblems for dynamic programming. **Solution.** Let, for all $i, j = 1, 2, \dots, n$ and $i \leq j$, $\text{ENERGY}(i, j)$ denote the optimal solution for the problem defined with liquids $v_i, v_{i+1}, \dots, v_j$.

Rubric: +1 for writing everything correctly. +0.5 for writing the subproblems correctly but missing the range of i, j . 0 for writing non-mathematical blurbs or pseudocodes.

- (b) [3 points] Write the recurrences for optimal substructure that handles all cases, including the base, final solution. **Solution.**

(a) *Base cases:* For all $i = 1, 2, \dots, n$, $\text{ENERGY}(i, i) = v_i$. **Rubric:** +0.5 for base case

(b) For all $1 \leq i \leq n, i < j \leq n$

$$\text{ENERGY}(i, j) = \max_{i \leq k < j} \{ \text{ENERGY}(i, k) + \text{ENERGY}(k+1, j) + \text{SUM}(v_i, v_{i+1}, \dots, v_k)^2 + \text{SUM}(v_{k+1}, v_{k+2}, \dots, v_j)^2 \}$$

Rubric: +2 for writing the correct recurrence. -0.5 if recurrence is correct but range of indices i, j or k is wrong or unspecified

Final Solution. We want to compute $\text{ENERGY}(1, n)$. **Rubric:** +0.5 for correct answer, 0 for anything else.

- (c) [4 points] Write an **iterative** pseudocode on the basis of the above recurrence. Mention inputs, outputs and data-structures (like array etc.) properly. **Solution.**

Algorithm 4 Energy

```

1: procedure ENERGY( $(1, n)$ )
2:   ENERGY : 2-D Array of size  $(n) \times (n)$  Rubric: +1 for declaring 2-D array correctly
3:   for  $i = 1$  to  $n$  do
4:     ENERGY[ $i$ ][ $i$ ] =  $v_i$  Rubric: +1 for filling up the base-case entries
5:   end for
6:   for  $l = 1$  to  $n - 1$  do
7:     for  $i = 1$  to  $n - l$  do
8:        $j = i + l$ 
9:       for  $k = i$  to  $j - 1$  do
10:        ENERGY[ $i$ ][ $j$ ] =  $\max\{\text{ENERGY}[i][k] + \text{ENERGY}[k+1][j] + (\text{SUM}(v_i, v_{i+1}, \dots, v_k)^2 + \text{SUM}(v_{k+1}, v_{k+2}, \dots, v_j)^2),$ 
11:          ENERGY[ $i$ ][ $j$ ]\}
12: Rubric: +2 for writing the loops in correct order and writing the max thing correctly. This is pretty much 0 or full -
    you cannot have a ‘partially’ correct pseudocode, so don’t argue !
13:     end for
14:   end for
15: end for
16:   Return ENERGY[1][ $n$ ]
17: end procedure

```

- (d) [2 points] Argue runtime.

Solution. The runtime is dominated by the three nested for loops, each of which runs for $\theta(n)$ many times. Hence, the runtime is $\theta(n^3)$

Rubric: +1 for the right answer (even $O(n)$ gets full credit) and +1 for reasoning

Remark: If your solutions to parts(a) and (b) are incorrect, missing or inconsistent with the pseudocode, you will not get *any credit* for the pseudocode (part c) or runtime (part d).

Problem 4. We consider a slight variant of the maximum independent set problem on the line. We have a path graph with n vertices $v_1, v_2, \dots, v_n$, where vertex v_i has non-negative weight w_i . The goal is to pick a maximum weight independent set of the path *with cardinality* k , where $k \leq n$ is a non-negative integer. Your algorithm should run in time $\theta(nk)$.

- (a) **[1 points]** Define the subproblems for dynamic programming.

Solution: $OPT(i, j)$ be the maximum weight of an independent set chosen from the prefix of vertices $v_1, v_2, \dots, v_i$ such that the chosen set contains at most j vertices.

Rubric: +1 for writing everything correctly. +0.5 for writing the subproblems correctly but missing the range of i, j . 0 for writing non-mathematical blurbs or pseudocodes.

- (b) **[3 points]** Write the recurrences for optimal substructure that handles all cases, including the base, final solution.

Solution: Base Case:

$$OPT(i, 0) = 0 \quad \text{for all } 0 \leq i \leq n$$

$$OPT(0, j) = 0 \quad \text{for all } 0 \leq j \leq k$$

$$OPT(1, j) = w_1 \quad \text{for all } 1 \leq j \leq k$$

Rubric: +1 for writing base cases correctly. +0.5 for correctly handling cases where cardinality is 0 ($OPT(i, 0) = 0$) and where no vertices are available ($OPT(0, j) = 0$). +0.5 for correctly handling the single vertex case ($OPT(1, j) = w_1$). Note: If the student uses other tricks allowing the main recurrence to handle $i = 1$ securely without a separate base case, award full points.

Recurrence:

$$OPT(i, j) = \max(OPT(i-1, j), OPT(i-2, j-1) + w_i) \quad \text{for all } 2 \leq i \leq n \text{ and } 1 \leq j \leq k$$

Rubric: +1.5 for writing the recurrence correctly. +0.5 for correctly writing the "exclude" choice: $OPT(i-1, j)$. +1 for correctly writing the "include" choice: $OPT(i-2, j-1) + w_i$. (-0.5 if $i-1$ used instead of $i-2$, violating the independent set constraint. -0.5 if students forget to decrement the cardinality j).

Final Solution:

$$OPT(n, k)$$

- (c) [4 points] Write an **iterative** pseudocode on the basis of the above recurrence. Mention inputs, outputs and data-structures (like array etc.) properly.

Rubric: +0.5 for writing the final solution correctly.

Algorithm 5 Maximum Weight Bounded Independent Set

```
1: procedure MAXWEIGHTINDEPENDENTSET( $w, n, k$ )
2:    $OPT$  : 2-D array of size  $(n + 1) \times (k + 1)$ 
3:   for  $i = 0$  to  $n$  do
4:      $OPT(i, 0) = 0$ 
5:   end for
6:   for  $j = 0$  to  $k$  do
7:      $OPT(0, j) = 0$ 
8:   end for
9:   for  $j = 1$  to  $k$  do
10:     $OPT(1, j) = w_1$ 
11:  end for
12:  for  $i = 2$  to  $n$  do
13:    for  $j = 1$  to  $k$  do
14:       $OPT(i, j) = \max(OPT(i - 1, j), OPT(i - 2, j - 1) + w_i)$ 
15:    end for
16:  end for
17:  Return  $OPT(n, k)$ 
18: end procedure
```

Rubric: +0.5 points for Initialization: Correctly allocates a 2D array (or equivalent data structure) of size $(n + 1) \times (k + 1)$. +1.0 for Base Case Setup as per their defined base cases in Part (b). +2.0 for the main loop Structure that translates the recurrence from Part (b). +0.5 for Output.

- (d) [2 points] Argue runtime.

Solution: The base cases are initialized using single loops that take $O(n + k)$ time. The core of the algorithm consists of two nested loops that execute exactly $(n - 1) \times k$ times. Because every operation inside these loops (array access, addition, and comparison) takes constant $O(1)$ time, the total runtime is strictly $\Theta(nk)$.

Rubric: +1 for correctly identifying the number of subproblems from loop and +1 for writing operations inside the loops are constants.